

UNIVERSIDAD TECNOLÓGICA NACIONAL

Tecnicatura Universitaria en Programación a Distancia

TRABAJO PRÁCTICO N.º 1

**Del dominio al esquema: modelo entidad-relación,
modelo relacional, normalización y DDL**

Materia	Base de Datos I
Proyecto	Food Store - sistema de gestión de pedidos
Alumno	Nicolás Moron
Fecha	09/09/2026

Resolución integral

Incluye modelo ER, derivación relacional, normalización 1FN-BCNF, DDL PostgreSQL y declaración de uso de IA.

Contenido de la entrega

- Parte 1 - Modelo entidad-relación: entidades, atributos, claves, cardinalidades, participación y preguntas guía.
- Parte 2 - Modelo relacional: tablas derivadas, PK/FK y justificación de la tabla intermedia.
- Parte 3 - Normalización: clave candidata, dependencias funcionales, 1FN, 2FN, 3FN y BCNF, más integración.
- Parte 4 - DDL PostgreSQL: schema.sql ejecutable con tipos, PK, FK, UNIQUE, CHECK, DEFAULT e índices.
- Anexo - Declaración de Uso de IA (DUIA).

Criterio de diseño general

La resolución distingue el modelo conceptual del dominio de la normalización de la planilla histórica. El esquema definitivo de la Parte 4 concilia ambos resultados, tal como pide la consigna.

Parte 1 - Modelo entidad-relación

Se adopta una representación tipo **Crow's Foot** mediante una entidad asociativa **DETALLE_PEDIDO** para resolver la relación N:M entre PEDIDO y PRODUCTO. En las multiplicidades, 0 indica participación opcional y 1 indica participación obligatoria.

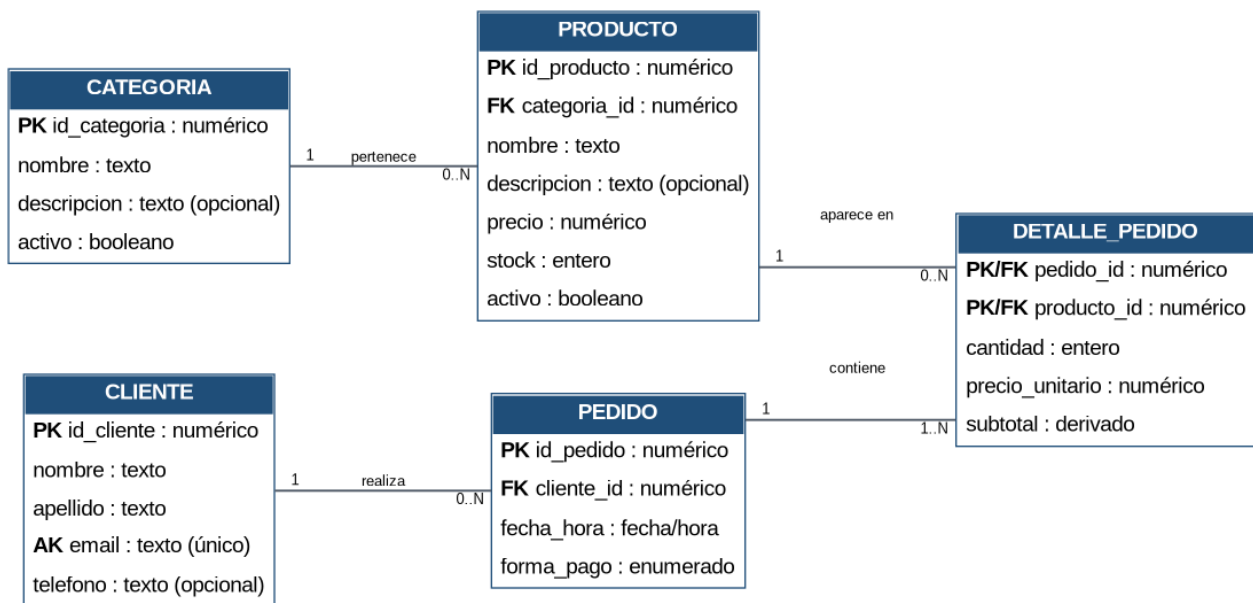


Figura 1. Modelo ER propuesto para Food Store. El subtotal se muestra como atributo derivado, no persistido en el esquema normalizado.

1.1 Diccionario de entidades y atributos

Entidad	Atributo	Clave	Tipo conceptual	Justificación
CATEGORIA	id_categoria	PK	Númerico	Identificador interno.
CATEGORIA	nombre	-	Texto	Nombre visible de la categoría.
CATEGORIA	descripcion	-	Texto, opcional	Atributo adicional razonable.
CATEGORIA	activo	-	Booleano	Baja lógica; conserva historial.

Entidad	Atributo	Clave	Tipo conceptual	Justificación
PRODUCTO	id_producto	PK	Numérico	Identificador interno.
PRODUCTO	categoria_id	FK	Numérico	Categoría obligatoria del producto.
PRODUCTO	nombre	-	Texto	Nombre comercial.
PRODUCTO	descripcion	-	Texto, opcional	Atributo adicional razonable.
PRODUCTO	precio	-	Numérico	Precio de lista actual; no negativo.
PRODUCTO	stock	-	Entero	Existencias actuales; no negativas.
PRODUCTO	activo	-	Booleano	Baja lógica; conserva historial.
CLIENTE	id_cliente	PK	Numérico	Identificador interno.
CLIENTE	nombre	-	Texto	Nombre del cliente.
CLIENTE	apellido	-	Texto	Apellido del cliente.
CLIENTE	email	AK	Texto	Clave candidata alternativa; único.
CLIENTE	telefono	-	Texto, opcional	Atributo adicional razonable.
PEDIDO	id_pedido	PK	Numérico	Identificador del pedido.
PEDIDO	cliente_id	FK	Numérico	Cliente registrado obligatorio.
PEDIDO	fecha_hora	-	Fecha/hora	Momento del pedido.
PEDIDO	forma_pago	-	Enumerado	EFFECTIVO, TARJETA o TRANSFERENCIA.
DETALLE_PEDIDO	pedido_id	PK/FK	Numérico	Parte de la clave compuesta.
DETALLE_PEDIDO	producto_id	PK/FK	Numérico	Parte de la clave compuesta.
DETALLE_PEDIDO	cantidad	-	Entero	Unidades pedidas; debe ser > 0.
DETALLE_PEDIDO	precio_unitario	-	Numérico	Precio histórico de esa línea.
DETALLE_PEDIDO	subtotal	Derivado	Numérico	cantidad x precio_unitario; se calcula.

1.2 Cardinalidades y participación

Relación	Cardinalidad	Participación	Fundamento
CATEGORIA - PRODUCTO	1 : N	CATEGORIA parcial; PRODUCTO total	R1: una categoría puede tener 0..N productos; cada producto pertenece exactamente a una categoría.
CLIENTE - PEDIDO	1 : N	CLIENTE parcial; PEDIDO total	R2: un cliente puede tener 0..N pedidos; cada pedido pertenece a un cliente registrado.

Relación	Cardinalidad	Participación	Fundamento
PEDIDO - PRODUCTO	N : M	PEDIDO total en la versión final; PRODUCTO parcial	R3/R4: se resuelve con DETALLE_PEDIDO, que además guarda cantidad y precio_unitario.
PEDIDO - DETALLE_PEDIDO	1 : N	DETALLE total respecto de PEDIDO	Cada detalle no puede existir sin un pedido. Se asume que un pedido confirmado tiene al menos una línea.
PRODUCTO - DETALLE_PEDIDO	1 : N	DETALLE total; PRODUCTO parcial	Cada detalle referencia un producto; un producto puede no haber sido vendido aún.

1.3 Respuestas a las preguntas guía

¿Por qué PRODUCTO-PEDIDO no puede ser una relación 1:N?

Porque ambas direcciones admiten múltiples ocurrencias: un pedido puede contener varios productos y el mismo producto puede aparecer en muchos pedidos. Forzar 1:N haría falsa una de esas dos reglas. Además, la relación tiene atributos propios - **cantidad** y **precio_unitario** - que no pertenecen exclusivamente ni al pedido ni al producto. Por eso se materializan en DETALLE_PEDIDO.

¿Qué participación es parcial y cuál total en CATEGORIA-PRODUCTO?

CATEGORIA tiene participación parcial: puede existir una categoría recién creada sin productos. PRODUCTO tiene participación total: no puede existir sin pertenecer exactamente a una categoría. Si se invirtiera la lectura, se obligaría a toda categoría a tener al menos un producto y, al mismo tiempo, se permitirían productos sin categoría, contradiciendo R1.

¿Conviene descomponer algún atributo?

Sí. Se separa el nombre completo del cliente en **nombre** y **apellido** porque son datos que suelen consultarse, ordenarse o validarse por separado. Esto no significa que un nombre completo en una sola columna viole automáticamente 1FN: la decisión se adopta por utilidad semántica. En cambio, nombres de producto como "Coca 1.5L" se mantienen como un único atributo, ya que el enunciado no exige separar marca, presentación o volumen.

Parte 2 - Derivación al modelo relacional

Aplicando las reglas de derivación, cada entidad fuerte se convierte en tabla; las relaciones 1:N se implementan con una FK en el lado N y la relación N:M se resuelve mediante la tabla intermedia DETALLE_PEDIDO.

Tabla	Clave primaria	Claves foráneas	Otros atributos
categoria	id_categoria	-	nombre, descripcion, activo
producto	id_producto	categoria_id -> categoria	nombre, descripcion, precio, stock, activo
cliente	id_cliente	-	nombre, apellido, email, telefono
pedido	id_pedido	cliente_id -> cliente	fecha_hora, forma_pago
detalle_pedido	(pedido_id, producto_id)	pedido_id -> pedido; producto_id -> producto	cantidad, precio_unitario

2.1 Notación relacional

```
categoria (id_categoria, nombre, descripcion, activo)
```

```

producto (id_producto, categoria_id -> categoria, nombre, descripcion, precio, stock,
activo)

cliente (id_cliente, nombre, apellido, email, telefono)

pedido (id_pedido, cliente_id -> cliente, fecha_hora, forma_pago)

detalle_pedido (pedido_id -> pedido, producto_id -> producto, cantidad,
precio_unitario), PK = (pedido_id, producto_id)

```

2.2 Elección de la clave en DETALLE_PEDIDO

Se elige la clave primaria compuesta (**pedido_id, producto_id**). La regla de negocio indica que, dentro de un mismo pedido, un producto no se repite en más de una línea. Por lo tanto, esa combinación identifica naturalmente una línea y, además, impide duplicados sin necesitar una clave sustituta adicional. Si en el futuro se necesitara identificar líneas independientemente, podría agregarse un id técnico, manteniendo igualmente UNIQUE(pedido_id, producto_id).

2.3 Respuestas a las preguntas guía

¿Qué ocurriría si las dos FK de la tabla intermedia no fueran NOT NULL?

Podrían existir líneas incompletas: un detalle sin pedido o sin producto. Eso impediría interpretar la fila como la asociación entre ambos extremos, rompería la identidad de la clave compuesta y permitiría datos sin trazabilidad. En el modelo propuesto ambas FK son obligatorias y forman la PK, por lo que no admiten NULL.

¿Qué cambia si un pedido recién iniciado puede existir sin productos?

Cambiaría la participación de PEDIDO en la relación con DETALLE_PEDIDO de total a parcial: un pedido podría tener 0..N detalles. El modelo relacional no necesita FK nullable ni cambios estructurales; simplemente se permite insertar un PEDIDO sin filas hijas en DETALLE_PEDIDO. La participación de cada DETALLE_PEDIDO respecto de PEDIDO y PRODUCTO seguiría siendo total.

Parte 3 - Normalización hasta 3FN/BCNF

Se parte de la relación universal de la planilla histórica:

```

VENTA_PLANA(nro_pedido, fecha, cliente, producto, categoria, precio_unitario, cantidad,
subtotal, forma_pago)

```

Paso 1 - Clave candidata

La clave candidata mínima es (**nro_pedido, producto**). Un número de pedido no alcanza porque un pedido puede tener varias líneas. El producto solo tampoco alcanza porque puede aparecer en muchos pedidos. La combinación sí identifica una fila, dado que la regla indica que un producto no se repite dentro del mismo pedido.

Paso 2 - Dependencias funcionales

Dependencia funcional	Fundamento
nro_pedido -> fecha, cliente, forma_pago	Cada pedido tiene una única fecha, un único cliente y una única forma de pago.
producto -> categoria	Cada producto pertenece a una única categoría.
(nro_pedido, producto) -> cantidad	La línea identificada por pedido+producto fija la cantidad.
(nro_pedido, producto) -> precio_unitario	El precio histórico depende de la línea de pedido, no del producto por sí solo.
(cantidad, precio_unitario) -> subtotal	Por definición, subtotal = cantidad x precio_unitario.
(nro_pedido, producto) -> subtotal	Dependencia inferida por las anteriores.

Observación clave

No vale producto -> precio_unitario. La Muzzarella aparece a 1000 en un pedido y a 1050 en otro. Por eso el precio facturado debe quedar asociado a la línea histórica del pedido.

Paso 3 - Verificación de 1FN

La relación está en 1FN: cada celda contiene un único valor y no hay listas ni grupos repetidos dentro de una misma columna. Los textos de cliente y producto se tratan como valores atómicos para este ejercicio. Separar "Ana Gómez" en nombre y apellido puede ser útil en el modelo de negocio, pero no es condición necesaria para afirmar 1FN en esta planilla.

Paso 4 - Verificación y descomposición a 2FN

La clave es compuesta, por lo que se buscan dependencias parciales. Se encuentran dos: **nro_pedido -> fecha, cliente, forma_pago** y **producto -> categoria**. Esos atributos dependen de una sola parte de la clave, por lo que VENTA_PLANA no está en 2FN.

Tabla después de 2FN	PK	Atributos
PEDIDO_HIST	nro_pedido	fecha, cliente, forma_pago
PRODUCTO_HIST	producto	categoria
DETALLE_PEDIDO_HIST	(nro_pedido, producto)	cantidad, precio_unitario, subtotal

Paso 5 - Verificación y descomposición a 3FN

En DETALLE_PEDIDO_HIST, si subtotal se almacena, existe la dependencia **(cantidad, precio_unitario) -> subtotal**. El determinante no es una superclave y subtotal es un atributo no clave. Por lo tanto, subtotal introduce una dependencia entre atributos no clave y debe eliminarse del esquema lógico normalizado. Se calcula cuando se consulta la línea.

Tabla en 3FN	PK	Atributos no clave
PEDIDO_HIST	nro_pedido	fecha, cliente, forma_pago
PRODUCTO_HIST	producto	categoria
DETALLE_PEDIDO_HIST	(nro_pedido, producto)	cantidad, precio_unitario

Paso 6 - Verificación de BCNF

Tabla	FD relevante	Determinante	Resultado
PEDIDO_HIST	nro_pedido -> fecha, cliente, forma_pago	nro_pedido es clave candidata	Cumple BCNF
PRODUCTO_HIST	producto -> categoria	producto es clave candidata	Cumple BCNF
DETALLE_PEDIDO_HIST	(nro_pedido, producto) -> cantidad, precio_unitario	El determinante es la clave compuesta	Cumple BCNF

No quedan dependencias funcionales no triviales cuyo determinante no sea una clave candidata. Por eso el conjunto resultante está en BCNF.

Paso 7 - Conjunto final normalizado

PEDIDO_HIST (nro_pedido PK, fecha, cliente, forma_pago)

PRODUCTO_HIST (producto PK, categoria)

DETALLE_PEDIDO_HIST (nro_pedido PK/FK, producto PK/FK, cantidad, precio_unitario)

3.1 Preguntas de integración

Comparación entre normalización histórica y modelo ER

PEDIDO_HIST corresponde a PEDIDO, PRODUCTO_HIST a PRODUCTO y DETALLE_PEDIDO_HIST a DETALLE_PEDIDO. Sin embargo, la normalización de esta planilla no obliga a crear tablas CLIENTE y CATEGORIA separadas porque allí solo aparecen sus nombres, sin atributos adicionales que generen nuevas dependencias. El modelo ER sí las trata como entidades porque el enunciado de negocio exige información y reglas propias: por ejemplo, email único del cliente, stock y precio actual del producto y baja lógica de productos/categorías.

Esto muestra que normalizar datos históricos y modelar el dominio no son la misma tarea. La normalización elimina redundancias y anomalías de los datos disponibles; el modelado de dominio incorpora reglas, entidades y datos que el sistema necesita aunque no estén presentes en la planilla original. El esquema definitivo debe conciliar ambos enfoques.

¿Conviene almacenar subtotal?

Para el esquema lógico normalizado conviene calcularlo como **cantidad x precio_unitario** y no persistirlo. Si se almacena como un dato independiente, aparece la FD (cantidad, precio_unitario) -> subtotal y se introduce redundancia, con riesgo de inconsistencia. En una implementación real podría persistirse como columna generada o caché por rendimiento, pero sería una denormalización controlada. La trazabilidad histórica se resuelve almacenando precio_unitario de la línea; no hace falta duplicar subtotal.

Parte 4 - DDL en PostgreSQL

El siguiente schema.sql implementa el esquema definitivo conciliando el ER con la normalización. Usa identificadores autogenerados, TIMESTAMPTZ, NUMERIC para montos, restricciones declarativas y ON DELETE RESTRICT para proteger el historial.

```
-- Food Store - TP1 - Base de Datos I
-- PostgreSQL

CREATE TYPE forma_pago_enum AS ENUM (
    'EFECTIVO',
    'TARJETA',
    'TRANSFERENCIA'
);

CREATE TABLE categoria (
    id_categoria BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    descripcion VARCHAR(255),
    activo BOOLEAN NOT NULL DEFAULT TRUE,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE cliente (
    id_cliente BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    nombre VARCHAR(80) NOT NULL,
    apellido VARCHAR(80) NOT NULL,
    email VARCHAR(254) NOT NULL UNIQUE,
    telefono VARCHAR(30),
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE producto (
    id_producto BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    categoria_id BIGINT NOT NULL,
    nombre VARCHAR(120) NOT NULL,
    descripcion VARCHAR(255),
    precio NUMERIC(12,2) NOT NULL,
    stock INTEGER NOT NULL DEFAULT 0,
    activo BOOLEAN NOT NULL DEFAULT TRUE,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),

    CONSTRAINT fk_producto_categoria
        FOREIGN KEY (categoria_id)
        REFERENCES categoria (id_categoria)
```

```

        ON DELETE RESTRICT,

        CONSTRAINT ck_producto_precio_no_negativo
            CHECK (precio >= 0),

        CONSTRAINT ck_producto_stock_no_negativo
            CHECK (stock >= 0)
    );

CREATE TABLE pedido (
    id_pedido BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    cliente_id BIGINT NOT NULL,
    fecha_hora TIMESTAMPTZ NOT NULL DEFAULT now(),
    forma_pago forma_pago_enum NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),

    CONSTRAINT fk_pedido_cliente
        FOREIGN KEY (cliente_id)
        REFERENCES cliente (id_cliente)
        ON DELETE RESTRICT
);

CREATE TABLE detalle_pedido (
    pedido_id BIGINT NOT NULL,
    producto_id BIGINT NOT NULL,
    cantidad INTEGER NOT NULL,
    precio_unitario NUMERIC(12,2) NOT NULL,

    CONSTRAINT pk_detalle_pedido
        PRIMARY KEY (pedido_id, producto_id),

    CONSTRAINT fk_detalle_pedido_pedido
        FOREIGN KEY (pedido_id)
        REFERENCES pedido (id_pedido)
        ON DELETE RESTRICT,

    CONSTRAINT fk_detalle_pedido_producto
        FOREIGN KEY (producto_id)
        REFERENCES producto (id_producto)
        ON DELETE RESTRICT,

    CONSTRAINT ck_detalle_cantidad_positiva
        CHECK (cantidad > 0),

    CONSTRAINT ck_detalle_precio_no_negativo
        CHECK (precio_unitario >= 0)
);

-- Acelera la consulta de pedidos de un cliente ordenados por fecha.
CREATE INDEX idx_pedido_cliente_fecha
    ON pedido (cliente_id, fecha_hora DESC);

-- Acelera el listado/filtrado de productos por categoría y estado activo.
CREATE INDEX idx_producto_categoria_activo
    ON producto (categoria_id, activo);

-- Acelera consultas históricas: en qué pedidos apareció un producto.
CREATE INDEX idx_detalle_producto
    ON detalle_pedido (producto_id);

```

4.1 Justificación de decisiones DDL

Decisión	Justificación
NUMERIC(12,2)	Evita errores de punto flotante en importes monetarios.
TIMESTAMPTZ	Cumple el requisito de fecha/hora con zona horaria.
email UNIQUE	Implementa la clave candidata definida por R6.
CHECK precio >= 0	Implementa R5 para precio de lista.
CHECK stock >= 0	Implementa R5 para existencias.
CHECK cantidad > 0	Una línea de pedido debe representar al menos una unidad.

Decisión	Justificación
CHECK precio_unitario >= 0	Evita facturar líneas con precio negativo.
activo DEFAULT TRUE	Implementa baja lógica para producto y categoría, preservando historial.
ON DELETE RESTRICT	Impide borrar físicamente filas que ya estén referenciadas, evitando pérdida o ruptura de historial.
PK compuesta en detalle	Enforce que un producto no se repita en un mismo pedido.

4.2 Comprobación contra los requisitos mínimos

- Tipos adecuados: BIGINT IDENTITY, VARCHAR, NUMERIC, INTEGER, BOOLEAN, TIMESTAMPTZ y ENUM.
- PK explícita en las cinco tablas.
- FK explícitas para CATEGORIA-PRODUCTO, CLIENTE-PEDIDO y las dos FK de DETALLE_PEDIDO.
- UNIQUE en cliente.email.
- Cuatro restricciones CHECK: precio, stock, cantidad y precio_unitario.
- NOT NULL en todas las participaciones obligatorias; descripcion y telefono permanecen opcionales.
- DEFAULT en activo, stock, created_at y fecha_hora.
- Tres índices, cada uno con comentario de la consulta que acelera.
- No se almacena subtotal: se deriva de cantidad * precio_unitario.

Anexo - Declaración de Uso de IA (DUIA)

Herramienta utilizada: **ChatGPT (OpenAI)**.

Uso: análisis del enunciado, identificación de entidades y dependencias funcionales, contraste entre modelo ER y normalización, propuesta de DDL y revisión de restricciones de integridad. La resolución fue organizada para que cada decisión quede vinculada con las reglas del dominio y pueda verificarse manualmente antes de ejecutar el script.

Prompt/spec principal registrado:

"Resolver el TP1 Food Store respetando las reglas R1-R7: construir el modelo ER, derivar el modelo relacional, normalizar la planilla paso a paso hasta 3FN/BCNF y generar un schema.sql PostgreSQL ejecutable con PK, FK, UNIQUE, CHECK, DEFAULT e índices justificados."

Criterio de validación: no ejecutar código generado sin revisión. Antes de entregar, verificar el schema.sql en una base PostgreSQL vacía y revisar que las cardinalidades, FKs y restricciones coincidan con el modelo explicado en el documento.

Conclusión

La solución final separa correctamente categorías, productos, clientes, pedidos y líneas de pedido. La relación N:M entre pedidos y productos queda resuelta por DETALLE_PEDIDO, que conserva cantidad y precio_unitario histórico. La normalización de la planilla alcanza BCNF al eliminar dependencias parciales y no persistir subtotal. El DDL final incorpora las reglas de integridad exigidas y conserva el historial mediante baja lógica y restricciones de borrado.